The slow eigenspectrum of the training kernel

Trapped error, its scaling with network size, and when a learned time stepper should use a higher-order method

Draft

Abstract

A learned time stepper advances a dynamical system by one step with a neural network. The network can predict the next state directly, an increment as in the Euler step, or a higher time derivative combined with known lower derivatives, as in a Taylor method of order p. This note asks which of these a given network should use, and answers it through the training kernel of the network. The residual of a trained network decays at a rate equal to the Rayleigh quotient of the residual on the kernel, so the error that survives training is the part of the residual that sits on the slow eigenspectrum of the kernel. We define the hardness of a task as this trapped error, show that it is bounded below by the residual mass on eigenvalues smaller than the inverse training budget, and identify the quantities that determine it: the spectrum of the kernel, which depends on the network and the states, and the alignment of the target with that spectrum, which depends on the target. A Taylor method of order p has a target of size $h^p m_p/p!$ whose shape, after standardization, is that of the p-th time derivative and does not depend on h. Its trained error is therefore $h^p m_p\, \eta_p/p!$, where η_p is the unlearned fraction of the p-th derivative on the network. Higher derivatives are rougher functions of the state, so η_p increases with p and decreases with network size. We state a scaling ansatz for η_p as a function of size, show how its exponents are measured on small networks, and derive the size $N_p^*(h)$ above which, or the step $h_p^*(N)$ below which, order p has lower error than order $p-1$. Proofs are in Appendix B.

Contents

1 Introduction **2**

2 Setting **3**
2.1 System and data . . . 3
2.2 Network, methods, and protocol . . . 3

3 Kernel analysis of a training run **4**
3.1 Residual dynamics . . . 4
3.2 Decay rate and trapped error . . . 4
3.3 How the residual gets trapped: spectrum and alignment . . . 5
3.4 What to track . . . 6

4 Scaling of the tracked quantities **6**
4.1 Exact constraints . . . 6
4.2 Scaling ansatz . . . 7

5 Targets of time-stepping methods **7**
5.1 Taylor methods and the remainder lemma . . . 8
5.2 Why higher derivatives are rougher . . . 8
5.3 Imperfect lower slots and rollout . . . 9

6 When to switch order **9**
6.1 The trained error of a Taylor method . . . 9
6.2 The crossover . . . 10
6.3 Scaling of the crossover with network size . . . 10
6.4 Decision rules . . . 10

7 Items that need justification **11**

8 Tests **11**

A Notation **12**

B Proofs **12**
B.1 Proof of Proposition 1 . . . 12
B.2 Proof of Theorem 1 . . . 12
B.3 Proof of Lemma 1 . . . 13
B.4 Proof of Proposition 2 . . . 13
B.5 Proof of Lemma 2 and Corollary 1 . . . 13
B.6 Proof of Lemma 3 . . . 13
B.7 Proofs of Lemma 4 and Propositions 3 and 4 . . . 14
B.8 Proofs of Theorems 2 and 3 . . . 14

1 Introduction

A learned time stepper is a neural network trained to advance the state of a dynamical system by one time step. The same network can be used in several ways. In the direct step, the network outputs the next state. In the Euler step, it outputs an increment, and the next state is the current state plus the time step times this increment. In a Taylor method of order p, known lower time derivatives are combined with a network that outputs the p-th derivative. Each choice gives the network a different target to learn. The goal of this note is to say, for a given network and dataset, which choice gives the lower error, and how the answer changes as the network grows.

The argument has three parts. Section 3 analyzes one training run through the training kernel of the network. The residual decays at a rate set by where it sits on the spectrum of the kernel, and the error that survives training is the residual mass that sits on the slow part of the spectrum. This is the hardness of the task. It is determined by two things: the spectrum of the kernel, which is a property of the network and the states, and the alignment of the target with the eigenvectors of the kernel, which is a property of the target. The section ends with a list of the quantities that must be tracked during a training run to see both.

Section 4 asks how these quantities change with the size of the network, the amount of data, and the training budget. The exact statements available are constraints: the rank of the kernel is at most the number of parameters, so a small network has an exact null space in which no error decays; and the spectrum per training state converges as the data grow, so data act on generalization and

not on the training residual. Within these constraints, the dependence on size is a scaling ansatz whose exponents are measured on small networks.

Section 5 describes the targets of time-stepping methods. The target of order p has size proportional to h^p, and its shape after standardization is that of the p-th time derivative, independent of h. For a polynomial vector field the p-th derivative is a polynomial of degree that grows with p, which is why it is rougher and lies further along the slow spectrum. Section 6 combines the three parts. The trained error of order p is $h^p m_p \eta_p / p!$, with η_p the unlearned fraction of the p-th derivative, and the crossover between orders $p-1$ and p is at $h_p^*(N) = p\,(m_{p-1}\eta_{p-1})/(m_p\eta_p)$. Under the scaling ansatz this gives the network size above which order p is the better choice. Section 7 lists what needs justification, and Section 8 lists the experiments.

2 Setting

2.1 System and data

The system is $\dot{x} = f(x)$ with $x \in \mathbb{R}^{n_x}$. The flow map Φ_t sends a state to the state a time t later. The data are pairs $(x, \Phi_h(x))$ computed by a numerical solver with time step h. The solver error is neglected, so the data are noiseless. The vector field f is available to the solver that generates the data; it may or may not be available to the time stepper.

The time derivatives of a trajectory are functions of the state. Set $x^{(0)}(x) = x$, $x^{(1)} = f$, and $x^{(k+1)} = Dx^{(k)}\, f$, so that the k-th time derivative at state x is $x^{(k)}(x)$.

Assumption 1 (Regularity)**.** The states lie in a compact set $\mathcal{A}$ that is invariant under the flow, $\Phi_t(\mathcal{A}) = \mathcal{A}$ for all $t \in \mathbb{R}$. For every order k used below, $x^{(k)}$ is bounded and Lipschitz on $\mathcal{A}$, with $M_k = \sup_{\mathcal{A}} \|x^{(k)}\|$, Lipschitz constant L_k, and $L = L_1$. Since $x^{(0)}$ is the identity, $L_0 = 1$.

Assumption 2 (Sampling)**.** The flow on $\mathcal{A}$ has an ergodic invariant probability measure μ, and training and validation states are sampled from μ.

For a chaotic dissipative system, $\mathcal{A}$ is the attractor and μ its physical measure. Assumption 2 holds for states taken from long trajectories after the transient, by Birkhoff's theorem, and is the standard working assumption for turbulence, the Kuramoto–Sivashinsky equation, and weather. For a function u on $\mathcal{A}$ we write $\|u\|_\infty = \sup_{\mathcal{A}} \|u\|$, $\|u\|_\mu = (\int \|u\|^2 d\mu)^{1/2}$, and $m_k = \|x^{(k)}\|_\mu \le M_k$.

2.2 Network, methods, and protocol

The network $\mathcal{N}_\theta$ has parameters $\theta \in \mathbb{R}^P$ and a width N; size is measured by N or by P as convenient. A method is a step map $G(x) = Q_h(x) + \sigma\, \mathcal{N}_\theta(x)$, where Q_h is a known part and σ a scale. The target of the method is $V = \Phi_h - Q_h$, the function the network output must equal, up to the scale, for the step map to be exact. The methods themselves are defined in Section 5; until then only V matters.

Methods are compared under the following protocol.

P1. The architecture, size, and initialization are the same for every method.

P2. The network input is the current state only.

P3. Each method trains on V/σ, with σ the root mean square of V over the training states, and the step map multiplies the network output by σ.

P4. The final linear layer is initialized to zero, so the initial network output is zero.

P5. The training procedure and budget are the same for every method.

P3 gives every method a network target of unit root mean square, and P4 makes the initial residual equal to minus the target.

Let $x_1, \dots, x_n$ be the training states and $y_1, \dots, y_m$ the validation states. For a function u, $u_{\mathrm{tr}} \in \mathbb{R}^{n n_{\mathrm{x}}}$ stacks $u(x_1), \dots, u(x_n)$, with inner product $\langle a, b \rangle_{\mathrm{tr}} = \frac{1}{n} a^\top b$ and norm $\|u\|_{\mathrm{tr}} = \langle u_{\mathrm{tr}}, u_{\mathrm{tr}} \rangle_{\mathrm{tr}}^{1/2}$; u_{va}, $\langle \cdot, \cdot \rangle_{\mathrm{va}}$, and $\| \cdot \|_{\mathrm{va}}$ are defined likewise on the validation states. Operator norms are taken with respect to these norms. By Assumption 2, $\|u\|_{\mathrm{tr}}$ and $\|u\|_{\mathrm{va}}$ approximate $\|u\|_\mu$ for large samples. The residual of a method in state units is $r = G - \Phi_h$.

3 Kernel analysis of a training run

3.1 Residual dynamics

The network is trained on the mean squared error over the training states. Following [2], the optimizer Adam is modeled as gradient flow preconditioned by a diagonal matrix $\Pi(t)$ with positive entries, t the training time; minibatch noise and momentum are neglected. Let $\mathcal{J}_{\mathrm{tr}}(t)$ and $\mathcal{J}_{\mathrm{va}}(t)$ be the Jacobians of the stacked network outputs on the training and validation states with respect to θ. The training kernel and the validation kernel are

$$A_{\mathrm{tr}}(t) = \frac{1}{n} \mathcal{J}_{\mathrm{tr}} \Pi \mathcal{J}_{\mathrm{tr}}^\top, \qquad A_{\mathrm{va}}(t) = \frac{1}{n} \mathcal{J}_{\mathrm{va}} \Pi \mathcal{J}_{\mathrm{tr}}^\top. \tag{1}$$

The training kernel is symmetric and positive semidefinite, with eigenvalues $\lambda_1(t) \geq \lambda_2(t) \geq \cdots \geq 0$ and orthonormal eigenvectors $u_i(t)$ in the inner product $\langle \cdot, \cdot \rangle_{\mathrm{tr}}$.

Proposition 1 (Residual dynamics)**.** *Under the protocol,*

$$\dot{r}_{\mathrm{tr}} = -A_{\mathrm{tr}}(t)\, r_{\mathrm{tr}}, \qquad \dot{r}_{\mathrm{va}} = -A_{\mathrm{va}}(t)\, r_{\mathrm{tr}}, \qquad r_{\mathrm{tr}}(0) = -V_{\mathrm{tr}}, \quad r_{\mathrm{va}}(0) = -V_{\mathrm{va}}. \tag{2}$$

With $\mathcal{U}(t, s)$ the solution operator of $\dot{r} = -A_{\mathrm{tr}} r$ and T the training time,

$$r_{\mathrm{tr}}(T) = -\mathcal{U}(T, 0)\, V_{\mathrm{tr}}, \qquad r_{\mathrm{va}}(T) = -\big(V_{\mathrm{va}} - \mathcal{K}_T V_{\mathrm{tr}}\big), \qquad \mathcal{K}_T = \int_0^T A_{\mathrm{va}}(t)\, \mathcal{U}(t, 0)\, dt. \tag{3}$$

Proposition 1 is proved in Appendix B.1. It follows from the chain rule and does not linearize the network. The kernel changes during training because the parameters change, and the path of the kernel depends on the target.

3.2 Decay rate and trapped error

Write the residual in the eigenbasis of the current kernel, $\hat{r}_i(t) = \langle u_i(t), r_{\mathrm{tr}}(t) \rangle_{\mathrm{tr}}$.

Theorem 1 (Decay rate and trapped error)**.** *Let $\rho(t) = \langle r_{\mathrm{tr}}, A_{\mathrm{tr}} r_{\mathrm{tr}} \rangle_{\mathrm{tr}} / \|r_{\mathrm{tr}}\|_{\mathrm{tr}}^2$ be the Rayleigh quotient of the residual on the kernel.*

(a) The residual decays at rate ρ:

$$\frac{d}{dt} \|r_{\mathrm{tr}}\|_{\mathrm{tr}}^2 = -2\, \rho(t)\, \|r_{\mathrm{tr}}\|_{\mathrm{tr}}^2, \qquad \rho(t) = \frac{\sum_i \lambda_i(t)\, \hat{r}_i(t)^2}{\sum_i \hat{r}_i(t)^2}. \tag{4}$$

(b) For a budget $\tau > 0$, define the trapped error at time t as the residual mass on eigenvalues at most $1/\tau$,

$$\mathcal{H}_\tau(t)^2 = \sum_{i:\ \lambda_i(t) \le 1/\tau} \hat{r}_i(t)^2. \tag{5}$$

Then, with $d_t(s) = \|(A_{\mathrm{tr}}(s) - A_{\mathrm{tr}}(t))\, r_{\mathrm{tr}}(s)\|_{\mathrm{tr}}$ the drift of the kernel along the residual,

$$\|r_{\mathrm{tr}}(t+\tau)\|_{\mathrm{tr}} \ \ge\ e^{-1}\,\mathcal{H}_\tau(t) - \int_t^{t+\tau} d_t(s)\, ds. \tag{6}$$

(c) The validation residual moves at a rate bounded by the square root of the training decay rate,

$$\|\dot{r}_{\mathrm{va}}\|_{\mathrm{va}} \le \kappa_{\mathrm{va}}(t)\,\sqrt{\rho(t)}\,\|r_{\mathrm{tr}}\|_{\mathrm{tr}}, \qquad \kappa_{\mathrm{va}} = \frac{\|\mathcal{J}_{\mathrm{va}}\Pi^{1/2}\|}{\sqrt{m}}. \tag{7}$$

Theorem 1 is proved in Appendix B.2. Part (a) says that the residual decays quickly while it sits on large eigenvalues and slowly once it sits on small ones. Since the fast components are removed first, $\rho(t)$ decreases through training and the residual migrates to the slow eigenspectrum. Part (b) is a lower bound on the error: whatever happens in the next τ units of training, the error cannot fall below the mass that currently sits on eigenvalues below $1/\tau$, except by rotating the kernel, which is what the drift term measures. A network whose kernel is nearly constant late in training cannot remove this error at all. Part (c) says that the validation residual stops moving when the training residual is trapped, so the trapped error is also what is left on the validation states.

Definition 1 (Hardness)**.** The hardness of a task, given by a target V, a network, and training states, is the trapped error $\mathcal{H}_\tau(T)$ at the end of training for a budget τ comparable to T. The unlearned fraction of the task is $\eta(V) = \|r_{\mathrm{tr}}(T)\|_{\mathrm{tr}}/\|V\|_{\mathrm{tr}}$, and $\eta_{\mathrm{va}}(V) = \|r_{\mathrm{va}}(T)\|_{\mathrm{va}}/\|V\|_{\mathrm{va}}$ on the validation states.

When the residual is trapped, $\|r_{\mathrm{tr}}(T)\|_{\mathrm{tr}} \approx \mathcal{H}_T(T)$ and the unlearned fraction is the trapped fraction. The definition is a diagnostic of a trained network. It does not depend on the kernel at initialization, and it does not require the kernel to be constant.

3.3 How the residual gets trapped: spectrum and alignment

Theorem 1 describes the residual at the end. To explain how it got there, freeze the kernel at its mean over training, $\bar{A}_{\mathrm{tr}} = \frac{1}{T}\int_0^T A_{\mathrm{tr}}\, dt$, with eigenpairs $(\bar{\lambda}_i, \bar{u}_i)$, and write $\hat{V}_i = \langle \bar{u}_i, V_{\mathrm{tr}}\rangle_{\mathrm{tr}}$ for the alignment of the target with the mean kernel.

Lemma 1 (Filtered target)**.** *With $d(s) = \|(A_{\mathrm{tr}}(s) - \bar{A}_{\mathrm{tr}})\, r_{\mathrm{tr}}(s)\|_{\mathrm{tr}}$,*

$$\left\| r_{\mathrm{tr}}(T) + e^{-T\bar{A}_{\mathrm{tr}}} V_{\mathrm{tr}} \right\|_{\mathrm{tr}} \le \int_0^T d(s)\, ds, \qquad \left\| e^{-T\bar{A}_{\mathrm{tr}}} V_{\mathrm{tr}} \right\|_{\mathrm{tr}}^2 = \sum_i e^{-2T\bar{\lambda}_i}\,\hat{V}_i^2. \tag{8}$$

Lemma 1 is proved in Appendix B.3. Up to the drift, the residual at the end of training is the target filtered by the kernel: each component of the target along $\bar{u}_i$ is damped by $e^{-T\bar{\lambda}_i}$. The unlearned fraction is then

$$\eta(V)^2 \approx \frac{\sum_i e^{-2T\bar{\lambda}_i}\hat{V}_i^2}{\sum_i \hat{V}_i^2}, \tag{9}$$

the power of the target on eigenvalues below about $1/T$. Two things enter: the spectrum $\{\bar{\lambda}_i\}$, which is set by the network and the states and is the same for every target trained on them up to the drift, and the alignment $\{\hat{V}_i\}$, which is set by the target. A task is hard when the kernel has many small eigenvalues and the target has much of its power on them. The drift term is the certificate for this picture; it is recorded during training and, being weighted by the residual, it ignores changes of the kernel on directions that carry no error.

3.4 What to track

The following quantities describe a training run. All are computed from the Jacobians and the residual, which are available during training.

Q1. *Kernel spectrum.* The eigenvalues $\lambda_i(T)$ of the final kernel, and of the mean kernel. Summaries: the slow count $n_{\text{slow}} = \#\{i : \lambda_i \le 1/T\}$, and the tail exponent α if $\lambda_i \approx \lambda_1 i^{-\alpha}$ for large i.

Q2. *Target alignment.* The cumulative power of the target on the mean kernel, $C_V(\lambda) = \sum_{\bar{\lambda}_i \le \lambda} \hat{V}_i^2 / \|V\|_{\text{tr}}^2$, and its value at $\lambda = 1/T$, the slow power $\pi_{\text{slow}}(V) = C_V(1/T)$.

Q3. *Residual spectrum.* The components $\hat{r}_i(T)$ on the final kernel, the trapped error $\mathcal{H}_\tau(T)$, the decay rate $\rho(T)$, and the unlearned fractions $\eta(V)$ and $\eta_{\text{va}}(V)$.

Q4. *Drift certificate.* $\int_0^T d(s)\, ds$ of Lemma 1 relative to $\|r_{\text{tr}}(T)\|_{\text{tr}}$, and $\int_t^T d_t(s)\, ds$ of Theorem 1 for the t at which the residual was last a fixed multiple of its final value.

Q5. *Generalization.* $\eta_{\text{va}}(V)/\eta(V)$, and κ_{va} of Eq. (7).

Q1 is the property of the network and states; Q2 is the property of the target; Q3 is their combination and is the hardness; Q4 says whether Q1 and Q2 explain Q3; Q5 says whether the training residual is the right proxy for the validation error.

4 Scaling of the tracked quantities

The hardness of a task changes when the network grows, when more data are used, or when training is longer. This section states what can be said exactly, then states the scaling ansatz that the experiments of Section 8 test.

4.1 Exact constraints

Proposition 2 (Constraints on the spectrum)**.** *(a)* Rank. *At every t,* $\operatorname{rank} A_{\text{tr}}(t) \le P$. *If $P < nn_{\text{x}}$, at least $nn_{\text{x}} - P$ eigenvalues are exactly zero, and the component of the residual in* $\ker A_{\text{tr}}(t)$ *has decay rate zero at time t.*

(b) Data. *For a fixed parameter vector, the nonzero eigenvalues of A_{tr} are those of the $P \times P$ matrix $\Pi^{1/2}\big(\frac{1}{n}\mathcal{J}_{\text{tr}}^\top \mathcal{J}_{\text{tr}}\big)\Pi^{1/2}$, and $\frac{1}{n}\mathcal{J}_{\text{tr}}^\top \mathcal{J}_{\text{tr}} \to \int_{\mathcal{A}} \mathcal{J}_x^\top \mathcal{J}_x \, d\mu(x)$ as $n \to \infty$, where $\mathcal{J}_x$ is the Jacobian of the output at state x.*

(c) Budget. *$\mathcal{H}_\tau(t)$ is nonincreasing in τ, and by Lemma 1 the residual after training for a time T is, up to the drift, at least the target mass on $\bar{\lambda}_i \le 1/T$ times e^{-1}.*

Proposition 2 is proved in Appendix B.4. Part (a) separates two regimes. A network with fewer parameters than residual components, $P < nn_{\mathrm{x}}$, has an exact null space; error in it does not decay at all, and can be reached only when the null space rotates. A network with $P \geq nn_{\mathrm{x}}$ can have a full-rank kernel, and its trapped error sits on small but nonzero eigenvalues. The scaling of hardness with size is different in the two regimes, and experiments should say which one they are in. Part (b) says that, for a fixed network, the spectrum per training state converges as the data grow. Adding data therefore does not slow the training residual; it acts on generalization, through $\mathcal{K}_T$ and the validation kernel, and on the path of the parameters. Part (c) says that a longer budget lowers the threshold $1/T$ below which error is trapped; it does not remove the error on the slowest modes unless T exceeds their inverse eigenvalue.

4.2 Scaling ansatz

The trained error of a task is $\|V\|\,\eta(V)$. The size of the target does not depend on the network, so the scaling of the error with size is the scaling of the unlearned fraction. The exact constraints do not determine it: in the limit of infinite width the kernel stops changing during training and its spectrum stops depending on width, so η plateaus, while at finite width the kernel adapts to the target and the plateau is approached from above. The dependence on width below the plateau is what the experiments measure.

Ansatz 1 (Size scaling)**.** For a fixed training protocol and dataset, and for widths N in a range below the plateau, the unlearned fraction of a target V satisfies

$$\eta(V; N) \simeq a(V)\, N^{-\gamma(V)}, \tag{10}$$

and the same holds for η_{va} with its own prefactor and exponent.

The prefactor and exponent are properties of the target on the architecture. By Eq. (9), η is the target power below $1/T$ on the mean kernel, so $\gamma(V)$ measures how fast the slow part of the spectrum that carries V is lifted as the network grows. A target that sits further along the tail has more to gain from size and, if the tail is lifted uniformly, a larger exponent; whether it does is a finding, not an assumption. The exponents are measured on a sequence of small networks and extrapolated; the extrapolation is valid as long as the fitted range is below the plateau, which the tracked spectrum shows directly, since at the plateau the spectrum stops changing with N.

Remark (Data scaling). For kernel regression with a fixed kernel whose eigenvalues decay as $i^{-\alpha}$, the generalization error decays as a power of the number of samples with an exponent set by α and by the decay of the target's alignment coefficients. A target with more power on the tail has the slower data exponent. Part (b) of Proposition 2 says that this is where data enter; the quantity to track is the tail of C_V on the validation kernel, and the prediction is that targets that are hard in the sense of Section 3 are also the ones that need the most data. This note does not develop the data axis further.

5 Targets of time-stepping methods

The framework so far applies to any target. This section gives the targets of time-stepping methods and the two properties that decide their hardness: the size of the target, which carries the dependence on h, and its shape, which is that of a time derivative and is rougher for higher order.

5.1 Taylor methods and the remainder lemma

Definition 2 (Taylor method of order p)**.** Let $p \geq 0$ and let $F_k \approx x^{(k)}$, $k = 1, \dots, p-1$, be estimates of the lower time derivatives, with $F_0 = x^{(0)}$. The Taylor method of order p is

$$x_{j+1} = Q_h(x_j) + \sigma\, \mathcal{N}_\theta(x_j), \qquad Q_h = \sum_{k=0}^{p-1} c_k F_k, \qquad c_k = \frac{h^k}{k!}, \tag{11}$$

with target $V_p = \Phi_h - Q_h$. Order 0 is the direct step, $V_0 = \Phi_h$; order 1 is the Euler step, $V_1 = \Phi_h - \mathrm{id}$. If every lower slot is exact, $F_k = x^{(k)}$, the target is V_p^{ex}.

Lemma 2 (Taylor remainder along the flow)**.** *Under Assumption 1, for $s \in \mathbb{R}$ and $n \geq 0$,*

$$\Phi_s = \sum_{k=0}^{n} \frac{s^k}{k!}\, x^{(k)} + \mathcal{T}_n(s), \qquad \|\mathcal{T}_n(s)\|_\infty \leq \frac{M_{n+1}|s|^{n+1}}{(n+1)!}, \qquad \mathrm{Lip}_{\mathcal{A}}\left(\mathcal{T}_n(s)\right) \leq \frac{L_{n+1} e^{|s|L} |s|^{n+1}}{(n+1)!}, \tag{12}$$

and $\mathrm{Lip}_{\mathcal{A}}(\Phi_s) \leq e^{|s|L}$. Under Assumption 2 also $\|\mathcal{T}_n(s)\|_\mu \leq m_{n+1}|s|^{n+1}/(n+1)!$.

Corollary 1 (Size and shape of the target)**.** *For $p \geq 1$ with exact lower slots, $V_p^{\mathrm{ex}} = c_p\, x^{(p)} + \mathcal{T}_p(h)$. Hence*

$$\|V_p^{\mathrm{ex}}\|_\mu = c_p\, m_p\, (1 + O(h)), \qquad \frac{V_p^{\mathrm{ex}}}{\|V_p^{\mathrm{ex}}\|_\mu} = \frac{x^{(p)}}{m_p} + O(h), \tag{13}$$

where the $O(h)$ terms are bounded by $h\, m_{p+1}/((p+1)m_p)$ in $\|\cdot\|_\mu$. For the direct step, $V_0 = \mathrm{id} + hf + \mathcal{T}_1(h)$, and for the Euler step, $V_1 = hf + \mathcal{T}_1(h)$, so $\mathrm{Lip}_{\mathcal{A}}(V_1) \leq e^{hL} - 1$ while $e^{-hL} \leq \mathrm{Lip}_{\mathcal{A}}(V_0) \leq e^{hL}$.

Lemma 2 and Corollary 1 are proved in Appendix B.5. The corollary is the link to Section 3. The size of the target of order p is $c_p m_p$, which carries all the dependence on h to leading order. Its shape, which is what the network trains on under P3, is $x^{(p)}/m_p$ up to $O(h)$, and does not depend on h. So, to leading order, every Taylor method of order p poses the same training problem, whatever the time step, and its unlearned fraction is the unlearned fraction of the p-th derivative,

$$\eta_p = \eta\big(x^{(p)}\big), \qquad \eta\big(V_p^{\mathrm{ex}}\big) = \eta_p + O(h). \tag{14}$$

The direct step is the exception: its target is the identity plus a small correction, and its hardness is that of the identity, $\eta_0 = \eta(\mathrm{id})$, which is small for a network that learns the identity easily.

5.2 Why higher derivatives are rougher

Lemma 3 (Degree of the time derivatives)**.** *If f is a polynomial of degree $d \geq 2$ in the state, then $x^{(k)}$ is a polynomial of degree at most $k(d-1)+1$, and the bound is attained generically.*

Lemma 3 is proved in Appendix B.6. Most systems of interest have quadratic vector fields, $d = 2$: Lorenz, Galerkin truncations of the Kuramoto–Sivashinsky and Navier–Stokes equations, and the dry primitive equations. For these, $x^{(k)}$ has degree $k+1$. For a kernel that is a function of inner products of states, and for isotropic inputs, polynomials of higher degree lie in eigenspaces of smaller eigenvalue, with a decay that depends on the architecture. The states on an attractor are not isotropic, so this is a heuristic for the physical measure; the measured alignment $C_{x^{(k)}}$ of Q2 is the actual statement. The prediction is that $\pi_{\mathrm{slow}}(x^{(k)})$, and therefore η_k, increases with k.

5.3 Imperfect lower slots and rollout

When f is not available to the stepper, the lower derivatives are estimated. A lower slot is state-based if F_k depends on the current state only and history-based if it uses predicted states from earlier steps. A finite difference is state-based during training and history-based during rollout.

Lemma 4 (Finite-difference slot). *Let $F_1 = h^{-1}\sum_{i=0}^{q}\phi_i\,\Phi_{-ih}$ with $\sum_i \phi_i(-i)^k/k! = \delta_{k1}$ for $k = 0,\dots,q$. Then $F_1 - f = h^{-1}\sum_i \phi_i \mathcal{T}_q(-ih)$ and $\|F_1 - f\|_\infty \le C_q M_{q+1} h^q$ with $C_q = \sum_i |\phi_i|\, i^{q+1}/(q+1)!$. The two-point difference $(1,-1)$ has $q=1$, $C_1 = 1/2$; the three-point difference $(3/2,-2,1/2)$ has $q=2$, $C_2 = 1$.*

Proposition 3 (Leading target with imperfect slots). *Let $e_k = F_k - x^{(k)} = h^{p-k}b_k + O(h^{p-k+1})$ for $k = 1,\dots,p-1$. Then $V_p = h^p w_p + O(h^{p+1})$ with $w_p = x^{(p)}/p! - \sum_{k=1}^{p-1} b_k/k!$. For $p=2$, the two-point backward difference gives $w_2 = x^{(2)}$ and the three-point difference gives $w_2 = x^{(2)}/2$, the exact-slot value.*

Proposition 4 (Parasitic root of a history-based slot). *Apply the Taylor method of order 2 with $F_1 = (x_j - x_{j-1})/h$ in rollout to $\dot{x} = \lambda x$, with $z = h\lambda$ and the network output equal to its exact target. The rollout recurrence is $x_{j+1} = (e^z + e^{-z})x_j - x_{j-1}$, with roots e^z and e^{-z}.*

Lemma 4 and Propositions 3 and 4 are proved in Appendix B.7. A finite-difference slot of order q changes the target by $O(h^{q+1})$, so it keeps the order-p target when $q \ge p-1$. When $q = p-1$ it changes the leading term: the two-point slot doubles the size of the order-2 target and leaves its shape, hence η_2, unchanged. The parasitic root e^{-z} has modulus larger than one on decaying modes, since the history-based slot turns the method into the two-step recurrence of $\ddot{x} = \lambda^2 x$, which cannot distinguish forward from backward time. On a dissipative system, a slot used in rollout should be state-based.

6 When to switch order

6.1 The trained error of a Taylor method

Theorem 2 (Error of order p). *Let the Taylor method of order $p \ge 1$ with exact lower slots be trained under the protocol, and let the validation residual be measured in $\|\cdot\|_\mu$. Then*

$$\delta_p(h) = \frac{h^p}{p!}\, m_p\, \eta_{p,\mathrm{va}}\,\big(1 + O(h)\big), \tag{15}$$

where $\eta_{p,\mathrm{va}}$ is the validation unlearned fraction of the p-th derivative, and the $O(h)$ term is bounded by $h\, m_{p+1}/((p+1)m_p)$ times a constant that depends on the continuity of the training map. If the kernels are frozen at the mean kernels of a reference run, the training map is linear, the constant is $1 + \|G_T\|$ with $G_T = \int_0^T \bar{A}_{\mathrm{va}} e^{-t\bar{A}_{\mathrm{tr}}} dt$, and Eq. (15) holds with $\eta_{p,\mathrm{va}}$ replaced by $\|\mathcal{D}x^{(p)}\|_{\mathrm{va}}/\|x^{(p)}\|_{\mathrm{va}}$, $\mathcal{D}u = u_{\mathrm{va}} - G_T u_{\mathrm{tr}}$.

Theorem 2 is proved in Appendix B.8. It separates the trained error into three factors: the classical factor $h^p/p!$, the size m_p of the p-th derivative, and the unlearned fraction $\eta_{p,\mathrm{va}}$, which is the hardness of the p-th derivative on the network and does not depend on h. The first two are properties of the system and can be computed from solver data; the third is the tracked quantity Q3 of one training run at any small h. The frozen form is the version that predicts a method before training it, from the kernels of one reference run; the gap between it and the trained value is bounded in Appendix B.8 by the drift and by the difference between the reference kernels and the method's own.

6.2 The crossover

Theorem 3 (Crossover between orders)**.** *Under the conditions of Theorem 2, order p has lower error than order $p-1$ for*

$$h < h_p^* = p\,\frac{m_{p-1}}{m_p}\cdot\frac{\eta_{p-1,\mathrm{va}}}{\eta_{p,\mathrm{va}}}\qquad(1+O(h)),\tag{16}$$

and higher error above it. In the frozen form, with $\nu_k = \|\mathcal{D}x^{(k)}\|_{\mathrm{va}}$ and $\epsilon_p = (1+\|G_T\|)\,M_{p+1}h^{p+1}/(p+1)!$, the sufficient condition without remainder is $2(c_p\nu_p+\epsilon_p) < c_{p-1}\nu_{p-1}$.

Theorem 3 is proved in Appendix B.8. The threshold has two factors. The first, $p\,m_{p-1}/m_p$, is the classical Taylor condition that the next term be smaller than the last. The second, η_{p-1}/η_p, is the learnability ratio. By Section 5 it is below one, since the p-th derivative is rougher than the $(p-1)$-th, so the network moves the crossover to a smaller time step than the classical condition gives. For $p=1$ the theorem compares the Euler step with the direct step: $h_1^* = (m_0/m_1)\,(\eta_0/\eta_1)$, where η_0 is the unlearned fraction of the identity.

6.3 Scaling of the crossover with network size

Corollary 2 (Crossover under the size ansatz)**.** *Under Ansatz 1 for $x^{(p-1)}$ and $x^{(p)}$, with exponents γ_{p-1} and γ_p and prefactors a_{p-1} and a_p,*

$$h_p^*(N) = p\,\frac{m_{p-1}a_{p-1}}{m_p a_p}\,N^{\gamma_p-\gamma_{p-1}}.\tag{17}$$

If $\gamma_p > \gamma_{p-1}$, order p becomes viable at a given step h once the width exceeds

$$N_p^*(h) = \Big(\frac{h\,m_p a_p}{p\,m_{p-1}a_{p-1}}\Big)^{1/(\gamma_p-\gamma_{p-1})}.\tag{18}$$

If $\gamma_p \le \gamma_{p-1}$, the crossover step does not grow with size, and order p is viable at h either for all sizes in the range or for none.

The corollary is immediate from Theorem 3. It is the statement that a sequence of small networks determines when a larger one should switch order: the four numbers $a_{p-1}, a_p, \gamma_{p-1}, \gamma_p$ are fitted on the small networks, and $N_p^*(h)$ follows. The condition $\gamma_p > \gamma_{p-1}$ says that the rougher target gains more from size. Under Eq. (9) this is the statement that growing the network lifts the far tail of the spectrum, where $x^{(p)}$ sits, faster than it lifts the near tail, where $x^{(p-1)}$ sits. Whether this holds is test T4 of Section 8, and it is the empirical center of the note.

6.4 Decision rules

Rule 1 (Order for a given network)**.** Train the direct step, the Euler step, and order 2 once each at a small h, and record Q1–Q5. Read off η_0, η_1, η_2 and compute h_1^* and h_2^* from Eq. (16). Use the highest order whose threshold exceeds the intended time step.

Rule 2 (Order for a larger network)**.** Repeat Rule 1 on a sequence of sizes below the plateau, fit Ansatz 1, and use Eq. (18) to decide the order for the target size. Check that the fitted range is below the plateau by confirming that the spectrum Q1 still changes with N.

Rule 3 (Lower slots)**.** Use exact lower slots when f is available. Otherwise use a finite-difference slot of order at least $p-1$, prefer the three-point difference for $p=2$ since it keeps the exact-slot leading term, and never use a finite difference of predicted states in rollout on a dissipative system.

The validation error is a one-step error. It enters the rollout error through the amplification by the step map, as in Eq. (20) of [1]. This note does not treat rollout beyond Proposition 4.

7 Items that need justification

J1. The drift certificate Q4. The trapped-error picture holds to the extent that $\int d$ is small relative to the final residual. This is a measurement.

J2. The h-independence of η_p, Eq. (14). It is exact for frozen kernels and holds for trained networks if the training map is continuous in the standardized target; the $O(h)$ term of Theorem 2 is checked by training order p at several h and confirming $\delta_p \propto h^p$ with the same η_p.

J3. The size ansatz and the sign of $\gamma_p - \gamma_{p-1}$. In the infinite-width limit both exponents are zero, so the ansatz describes feature learning at finite width, and the extrapolation is valid only below the plateau.

J4. The rank regime. Small networks with $P < nn_{\mathrm{x}}$ have an exact null space, and their scaling may differ from that of networks with full-rank kernels. The experiments should span both regimes or state which one they are in.

J5. The model of Adam as preconditioned gradient flow, as in [2]. Everything in Section 3 holds for any positive diagonal $\Pi(t)$.

J6. The isotropy heuristic of Section 5. The alignment of $x^{(k)}$ with the kernel on attractor states is measured, not derived.

J7. Methods that place the network inside the stages of an integrator, and the link from the one-step root-mean-square error to the rollout error.

8 Tests

T1. *Trapping.* For one network and one target, track Q1–Q4 during training. Theorem 1 predicts that $\rho(t)$ decreases, that the residual spectrum concentrates on $\lambda_i \leq 1/t$, and that the lower bound of Eq. (6) holds with a small drift term.

T2. *Spectrum and alignment.* On one network, compute C_V of Q2 for $V = \mathrm{id}, f, x^{(2)}, x^{(3)}$ on the mean kernel of a reference run. Lemma 3 predicts that the slow power π_{slow} increases with the order of the derivative, and Eq. (9) predicts η from C_V.

T3. *h-independence.* Train order 1 and order 2 at several h. Theorem 2 predicts $\delta_p \propto h^p$ with an h-independent η_p, and Theorem 3 predicts the crossover of the two curves at h_2^*.

T4. *Size scaling.* Repeat T3 over a sequence of widths, fit Ansatz 1 for η_0, η_1, η_2, and compare the extrapolated $N_2^*(h)$ with the width at which order 2 actually overtakes order 1. Track Q1 to locate the plateau and the rank regime of Proposition 2(a).

T5. *Reference-run prediction.* Compute the frozen-form hardness of every method from one reference run and compare with the trained values. Appendix B.8 gives the gap in terms of the drift and the kernel difference.

T6. *Slots.* Order 2 with an exact slot, a two-point slot, and a three-point slot. Proposition 3 predicts error ratios 2 and 1 with the same η_2. In rollout on a dissipative system, Proposition 4 predicts growth along the parasitic root for the history-based slot.

A Notation

Symbol	Meaning
f, Φ_t, h	vector field, flow map over time t, time step
$\mathcal{A}$, μ	invariant set of the states, ergodic invariant measure
$x^{(k)}$, M_k, L_k, m_k	k-th time derivative as a function of state; its supremum, Lipschitz constant, and $L^2(\mu)$ norm
$\mathcal{T}_n(s)$	Taylor remainder of Φ_s after order n
$\mathcal{N}_\theta$, θ, P, N	network, parameters, their number, width
G, Q_h, σ, V	step map, known part, scale, target $\Phi_h - Q_h$
p, c_k, F_k, V_p, V_p^{ex}	order, $h^k/k!$, lower-slot estimates, target of order p, with exact slots
q, ϕ_i, C_q, b_k, w_p	finite-difference order, coefficients, constant; leading slot error; leading target function
x_i, y_i, n, m	training and validation states and their numbers
u_{tr}, $\langle\cdot,\cdot\rangle_{\mathrm{tr}}$, $\|\cdot\|_{\mathrm{tr}}$	stacked values, inner product $\frac{1}{n}a^\top b$, and norm on training states; likewise va
r, r_{tr}, r_{va}	residual $G - \Phi_h$ and its training and validation parts
t, T, τ	training time, total budget, look-ahead budget
$\Pi(t)$, $\mathcal{J}_{\mathrm{tr}}$, $\mathcal{J}_{\mathrm{va}}$	preconditioner of Adam; Jacobians of the outputs on training and validation states
A_{tr}, A_{va}, $\bar{A}_{\mathrm{tr}}$, $\bar{A}_{\mathrm{va}}$	training and validation kernels and their means over $[0,T]$
λ_i, u_i, $\hat{r}_i$	eigenvalues and eigenvectors of $A_{\mathrm{tr}}(t)$; residual components
$\bar{\lambda}_i$, $\bar{u}_i$, $\hat{V}_i$	eigenpairs of $\bar{A}_{\mathrm{tr}}$; target alignment
$\mathcal{U}(t,s)$, $\mathcal{K}_T$, G_T, $\mathcal{D}$	solution operator; exact and frozen response operators; frozen validation map
ρ, $\mathcal{H}_\tau$, d, d_t	decay rate, trapped error, drift of the kernel along the residual
$\eta(V)$, $\eta_{\mathrm{va}}(V)$, η_p	unlearned fractions; that of $x^{(p)}$
n_{slow}, α, C_V, π_{slow}, κ_{va}	tracked quantities Q1, Q2, Q5
$a(V)$, $\gamma(V)$, a_p, γ_p	prefactor and exponent of the size ansatz
δ_p, h_p^*, N_p^*, ν_k, ϵ_p	trained error of order p, crossover step, crossover width, frozen-form norms and remainder
λ, z	rate of a linear test equation, $z = h\lambda$

B Proofs

B.1 Proof of Proposition 1

Let $\tilde{e}_{\mathrm{tr}}$ and $\tilde{e}_{\mathrm{va}}$ be the stacked residuals of the network output against the standardized target V/σ. The loss is $\mathcal{L} = \frac{1}{2n}\|\tilde{e}_{\mathrm{tr}}\|^2$ with gradient $\frac{1}{n}\mathcal{J}_{\mathrm{tr}}^\top \tilde{e}_{\mathrm{tr}}$, and the preconditioned flow is $\dot{\theta} = -\frac{1}{n}\Pi\mathcal{J}_{\mathrm{tr}}^\top \tilde{e}_{\mathrm{tr}}$. By the chain rule, $\dot{\tilde{e}}_{\mathrm{tr}} = \mathcal{J}_{\mathrm{tr}}\dot{\theta} = -A_{\mathrm{tr}}\tilde{e}_{\mathrm{tr}}$ and $\dot{\tilde{e}}_{\mathrm{va}} = \mathcal{J}_{\mathrm{va}}\dot{\theta} = -A_{\mathrm{va}}\tilde{e}_{\mathrm{tr}}$. Since $G - \Phi_h = \sigma(\mathcal{N}_\theta - V/\sigma)$, $r = \sigma\tilde{e}$ obeys the same equations, and P4 gives $r(0) = -V$. Integrating $\dot{r}_{\mathrm{va}} = A_{\mathrm{va}}\mathcal{U}(t,0)V_{\mathrm{tr}}$ gives Eq. (3). □

B.2 Proof of Theorem 1

(a) $\frac{d}{dt}\|r_{\mathrm{tr}}\|_{\mathrm{tr}}^2 = 2\langle r_{\mathrm{tr}}, \dot{r}_{\mathrm{tr}}\rangle_{\mathrm{tr}} = -2\langle r_{\mathrm{tr}}, A_{\mathrm{tr}} r_{\mathrm{tr}}\rangle_{\mathrm{tr}}$, and in the eigenbasis $\langle r, Ar\rangle_{\mathrm{tr}} = \sum_i \lambda_i \hat{r}_i^2$.

(b) Fix t and write $A = A_{\mathrm{tr}}(t)$. For $s \geq t$, $\dot{r}_{\mathrm{tr}}(s) = -A\, r_{\mathrm{tr}}(s) - (A_{\mathrm{tr}}(s) - A)\, r_{\mathrm{tr}}(s)$, so by variation of constants

$$r_{\mathrm{tr}}(t+\tau) = e^{-\tau A} r_{\mathrm{tr}}(t) - \int_t^{t+\tau} e^{-(t+\tau-s)A}\big(A_{\mathrm{tr}}(s) - A\big) r_{\mathrm{tr}}(s)\, ds.$$

The integral has norm at most $\int_t^{t+\tau} d_t(s)\,ds$, since e^{-uA} is a contraction for $u \geq 0$. In the eigenbasis of A, $\|e^{-\tau A} r_{\mathrm{tr}}(t)\|_{\mathrm{tr}}^2 = \sum_i e^{-2\tau\lambda_i} \hat{r}_i^2 \geq e^{-2} \sum_{\lambda_i \leq 1/\tau} \hat{r}_i^2 = e^{-2}\mathcal{H}_\tau(t)^2$. The triangle inequality gives Eq. (6).

(c) $\|\dot{r}_{\mathrm{va}}\|_{\mathrm{va}} = m^{-1/2}\|A_{\mathrm{va}} r_{\mathrm{tr}}\| = m^{-1/2} n^{-1} \|\mathcal{J}_{\mathrm{va}} \Pi^{1/2} (\Pi^{1/2} \mathcal{J}_{\mathrm{tr}}^\top r_{\mathrm{tr}})\| \leq m^{-1/2} n^{-1} \|\mathcal{J}_{\mathrm{va}} \Pi^{1/2}\|\, \|\Pi^{1/2} \mathcal{J}_{\mathrm{tr}}^\top r_{\mathrm{tr}}\|$, and $\|\Pi^{1/2} \mathcal{J}_{\mathrm{tr}}^\top r_{\mathrm{tr}}\|^2 = n\, r_{\mathrm{tr}}^\top A_{\mathrm{tr}} r_{\mathrm{tr}} = n^2 \rho \|r_{\mathrm{tr}}\|_{\mathrm{tr}}^2$. □

B.3 Proof of Lemma 1

Let $D(t) = \mathcal{U}(t,0) - e^{-t\bar{A}_{\mathrm{tr}}}$. Then $\dot{D} = -\bar{A}_{\mathrm{tr}} D - (A_{\mathrm{tr}}(t) - \bar{A}_{\mathrm{tr}})\mathcal{U}(t,0)$ with $D(0) = 0$, so $D(T) V_{\mathrm{tr}} = -\int_0^T e^{-(T-s)\bar{A}_{\mathrm{tr}}} (A_{\mathrm{tr}}(s) - \bar{A}_{\mathrm{tr}}) \mathcal{U}(s,0) V_{\mathrm{tr}}\, ds = \int_0^T e^{-(T-s)\bar{A}_{\mathrm{tr}}} (A_{\mathrm{tr}}(s) - \bar{A}_{\mathrm{tr}})\, r_{\mathrm{tr}}(s)\, ds$, using $\mathcal{U}(s,0) V_{\mathrm{tr}} = -r_{\mathrm{tr}}(s)$. Since $e^{-u\bar{A}_{\mathrm{tr}}}$ is a contraction, $\|D(T) V_{\mathrm{tr}}\|_{\mathrm{tr}} \leq \int_0^T d(s)\, ds$, and $r_{\mathrm{tr}}(T) + e^{-T\bar{A}_{\mathrm{tr}}} V_{\mathrm{tr}} = -D(T) V_{\mathrm{tr}}$. The spectral identity is the eigendecomposition of $\bar{A}_{\mathrm{tr}}$. □

B.4 Proof of Proposition 2

(a) $A_{\mathrm{tr}} = \frac{1}{n}(\mathcal{J}_{\mathrm{tr}} \Pi^{1/2})(\mathcal{J}_{\mathrm{tr}} \Pi^{1/2})^\top$ has rank at most that of $\mathcal{J}_{\mathrm{tr}} \in \mathbb{R}^{n n_{\mathrm{x}} \times P}$. For $v \in \ker A_{\mathrm{tr}}(t)$, $\langle v, A_{\mathrm{tr}} v \rangle_{\mathrm{tr}} = 0$, so the component of r_{tr} along v contributes zero to ρ in Eq. (4). (b) $BB^\top$ and $B^\top B$ share nonzero eigenvalues, with $B = n^{-1/2} \mathcal{J}_{\mathrm{tr}} \Pi^{1/2}$, so the nonzero eigenvalues of A_{tr} are those of $\Pi^{1/2}(\frac{1}{n}\mathcal{J}_{\mathrm{tr}}^\top \mathcal{J}_{\mathrm{tr}})\Pi^{1/2}$. Since $\mathcal{J}_{\mathrm{tr}}^\top \mathcal{J}_{\mathrm{tr}} = \sum_i \mathcal{J}_{x_i}^\top \mathcal{J}_{x_i}$ and the x_i are sampled from μ, the law of large numbers gives the limit. (c) The sum in Eq. (5) runs over a set that shrinks as τ grows. The last claim is Lemma 1 with $e^{-2T\bar{\lambda}_i} \geq e^{-2}$ for $\bar{\lambda}_i \leq 1/T$. □

B.5 Proof of Lemma 2 and Corollary 1

Fix $x \in \mathcal{A}$. By invariance, $\Phi_t(x) \in \mathcal{A}$ for all t and $\frac{d^k}{dt^k}\Phi_t(x) = x^{(k)}(\Phi_t(x))$, so Taylor's theorem gives

$$\mathcal{T}_n(s)(x) = \int_0^s \frac{(s-t)^n}{n!}\, x^{(n+1)}\big(\Phi_t(x)\big)\, dt. \tag{19}$$

The integrand has norm at most $M_{n+1}|s-t|^n/n!$, which gives the supremum bound. For $x, x' \in \mathcal{A}$, $\Delta(t) = \|\Phi_t(x) - \Phi_t(x')\|$ satisfies $\Delta(t) \leq \Delta(0) + L\left|\int_0^t \Delta\right|$, and Grönwall's inequality gives $\Delta(t) \leq e^{|t|L}\|x - x'\|$, which is $\mathrm{Lip}_{\mathcal{A}}(\Phi_t) \leq e^{|t|L}$; subtracting Eq. (19) at x' from that at x and using this gives the Lipschitz bound. For the μ bound, Minkowski's integral inequality and the invariance $\|x^{(n+1)} \circ \Phi_t\|_\mu = m_{n+1}$ give $\|\mathcal{T}_n(s)\|_\mu \leq \left|\int_0^s \frac{|s-t|^n}{n!} dt\right| m_{n+1}$.

For the corollary, $V_p^{\mathrm{ex}} = \mathcal{T}_{p-1}(h) = c_p x^{(p)} + \mathcal{T}_p(h)$ by Eq. (12) with $n = p-1$ and $n = p$. Then $\big|\|V_p^{\mathrm{ex}}\|_\mu - c_p m_p\big| \leq \|\mathcal{T}_p(h)\|_\mu \leq c_p m_p \cdot h m_{p+1}/((p+1)m_p)$, and $\|V_p^{\mathrm{ex}}/\|V_p^{\mathrm{ex}}\|_\mu - x^{(p)}/m_p\|_\mu \leq 2\|\mathcal{T}_p(h)\|_\mu/(c_p m_p)$ by the triangle inequality for normalized vectors. For the direct and Euler steps, $\mathrm{Lip}_{\mathcal{A}}(V_1) = \mathrm{Lip}_{\mathcal{A}}(\int_0^h f \circ \Phi_t\, dt) \leq L\int_0^h e^{tL} dt = e^{hL} - 1$, and $\mathrm{Lip}_{\mathcal{A}}(\Phi_h) \geq e^{-hL}$ follows from $\|x - x'\| = \|\Phi_{-h}\Phi_h x - \Phi_{-h}\Phi_h x'\| \leq e^{hL}\|\Phi_h x - \Phi_h x'\|$. □

B.6 Proof of Lemma 3

By induction. $x^{(1)} = f$ has degree $d = 1\cdot(d-1)+1$. If $x^{(k)}$ has degree at most $k(d-1)+1$, then $Dx^{(k)}$ has degree at most $k(d-1)$, and $x^{(k+1)} = Dx^{(k)} f$ has degree at most $k(d-1)+d = (k+1)(d-1)+1$. The top-degree part of $x^{(k+1)}$ is $D(\text{top of } x^{(k)})\cdot(\text{top of } f)$, which vanishes only on a proper algebraic subset of coefficient space. □

B.7 Proofs of Lemma 4 and Propositions 3 and 4

Lemma 2 with $s = -ih$, $n = q$ gives $\Phi_{-ih} = \sum_{k\le q} \frac{(-ih)^k}{k!} x^{(k)} + \mathcal{T}_q(-ih)$; multiply by ϕ_i/h and sum. The order conditions leave $h^0 x^{(1)} = f$, so $F_1 - f = h^{-1}\sum_i \phi_i \mathcal{T}_q(-ih)$, and $\|\mathcal{T}_q(-ih)\|_\infty \le M_{q+1}(ih)^{q+1}/(q+1)!$ gives C_q. The stated coefficients satisfy the order conditions; $C_1 = 1/2$ and $C_2 = (2+4)/6 = 1$.

For Proposition 3, $Q_h = Q_h^{\mathrm{ex}} + \sum_{k=1}^{p-1} c_k e_k$ and $c_k h^{p-k} = h^p/k!$, so $V_p = c_p x^{(p)} + \mathcal{T}_p(h) - \sum_k c_k(h^{p-k} b_k + O(h^{p-k+1})) = h^p w_p + O(h^{p+1})$. Two-point slot: $\Phi_{-h} = \mathrm{id} - hf + \frac{h^2}{2} x^{(2)} + \mathcal{T}_2(-h)$, so $e_1 = (\mathrm{id} - \Phi_{-h})/h - f = -\frac{h}{2} x^{(2)} + O(h^2)$, $b_1 = -x^{(2)}/2$, and $w_2 = x^{(2)}/2 + x^{(2)}/2 = x^{(2)}$. Three-point slot: $e_1 = O(h^2)$ by Lemma 4, so $b_1 = 0$.

For Proposition 4, on the true trajectory $x_{j-1} = e^{-z} x_j$, so the known part is $x_j + (x_j - x_{j-1}) = (2 - e^{-z}) x_j$ and the exact network target is $(e^z - 2 + e^{-z}) x_j$. In rollout, $x_{j+1} = 2x_j - x_{j-1} + (e^z - 2 + e^{-z}) x_j = (e^z + e^{-z}) x_j - x_{j-1}$, with characteristic polynomial $(\xi - e^z)(\xi - e^{-z})$. □

B.8 Proofs of Theorems 2 and 3

Frozen form. Let $\mathcal{D}u = u_{\mathrm{va}} - G_T u_{\mathrm{tr}}$ with G_T from the reference mean kernels. $\mathcal{D}$ is linear and $\|\mathcal{D}u\|_{\mathrm{va}} \le \|u\|_{\mathrm{va}} + \|G_T\| \|u\|_{\mathrm{tr}}$. By Corollary 1, $V_p^{\mathrm{ex}} = c_p x^{(p)} + \mathcal{T}_p(h)$, so

$$\big|\, \|\mathcal{D} V_p^{\mathrm{ex}}\|_{\mathrm{va}} - c_p \nu_p \,\big| \le \|\mathcal{D}\mathcal{T}_p(h)\|_{\mathrm{va}} \le \epsilon_p, \tag{20}$$

with ϵ_p as in Theorem 3 (or with M_{p+1} replaced by m_{p+1} when the sample norms are replaced by $\|\cdot\|_\mu$). Dividing by $c_p m_p$ gives Eq. (15) in the frozen form with constant $1 + \|G_T\|$. For the crossover, $V_{p-1}^{\mathrm{ex}} = c_{p-1} x^{(p-1)} + c_p x^{(p)} + \mathcal{T}_p(h)$, so $\|\mathcal{D} V_{p-1}^{\mathrm{ex}}\|_{\mathrm{va}} \ge c_{p-1}\nu_{p-1} - c_p \nu_p - \epsilon_p$, while $\|\mathcal{D} V_p^{\mathrm{ex}}\|_{\mathrm{va}} \le c_p \nu_p + \epsilon_p$; the sufficient condition follows, and dropping the remainders gives the crossover at $c_p \nu_p = c_{p-1}\nu_{p-1}$, that is $h = p\,\nu_{p-1}/\nu_p$, which is Eq. (16) with $\nu_k = \|x^{(k)}\|_{\mathrm{va}}\, \eta_{k,\mathrm{va}}$.

Trained form. Under P3 the network trains on $V_p^{\mathrm{ex}}/\sigma_p$, and by Corollary 1 this is $x^{(p)}/m_p + O(h)$. Write Ψ for the map from a standardized target to the standardized validation residual at time T; the validation error of the method is $\sigma_p \|\Psi(V_p^{\mathrm{ex}}/\sigma_p)\|_{\mathrm{va}}$. If Ψ is Lipschitz with constant Λ near $x^{(p)}/m_p$, then $\big|\|\Psi(V_p^{\mathrm{ex}}/\sigma_p)\|_{\mathrm{va}} - \|\Psi(x^{(p)}/m_p)\|_{\mathrm{va}}\big| \le \Lambda \|V_p^{\mathrm{ex}}/\sigma_p - x^{(p)}/m_p\|_{\mathrm{va}} = O(h)$, and $\|\Psi(x^{(p)}/m_p)\|_{\mathrm{va}} = \eta_{p,\mathrm{va}}$ by definition. With $\sigma_p = c_p m_p (1 + O(h))$ this is Eq. (15), and the crossover follows as in the frozen case. For frozen kernels Ψ is the linear map $\mathcal{D}$ and $\Lambda = 1 + \|G_T\|$.

Gap between the forms. By Proposition 1, the trained validation error is $\|V_{\mathrm{va}} - \mathcal{K}_T V_{\mathrm{tr}}\|_{\mathrm{va}}$, so $\big|\, \delta_{\mathrm{va}} - \|\mathcal{D}V\|_{\mathrm{va}} \,\big| \le \|(\mathcal{K}_T - G_T) V_{\mathrm{tr}}\|_{\mathrm{va}} + \|(G_T - G_T^{\mathrm{own}}) V_{\mathrm{tr}}\|_{\mathrm{va}}$, where G_T^{own} uses the method's own mean kernels. The first term is bounded by $\int_0^T \|(A_{\mathrm{va}}(t) - \bar{A}_{\mathrm{va}}) r_{\mathrm{tr}}(t)\|_{\mathrm{va}} dt + \|\bar{A}_{\mathrm{va}}\| \int_0^T (T - s)\, d(s)\, ds$ by the argument of Appendix B.3 applied to r_{va}, and the second by $\big(T\|\bar{A}_{\mathrm{va}} - \bar{A}_{\mathrm{va}}^{\mathrm{ref}}\| + \frac{T^2}{2}\|\bar{A}_{\mathrm{va}}^{\mathrm{ref}}\|\, \|\bar{A}_{\mathrm{tr}} - \bar{A}_{\mathrm{tr}}^{\mathrm{ref}}\|\big)\|V\|_{\mathrm{tr}}$, using $\|e^{-tX} - e^{-tY}\| \le t\|X - Y\|$ for symmetric positive semidefinite X, Y. □

References

[1] C. Ainslie, P. Hassanzadeh, M. W. Mahoney, and A. Chattopadhyay. Eigenanalysis framework for autoregressive neural emulators of multi-scale chaotic dynamics. arXiv:2608.16084, 2026.

[2] C. Ainslie, C.-Y. Lai, and A. Chattopadhyay. The dynamics of spectral bias in operator learning. In preparation.